DOCUMENTACIÓN TÉCNICA

SystemParCan

Auditoría, documentación y explicación educativa del código

Sistema de gestión para la Parroquia de Katuete

Java 21 · Spring Boot 4.1.1 · PostgreSQL · HTML · CSS · JavaScript

Actualizado: 12/11/2026 · Incluye errores por campo y validaciones de fechas

Contenido

1. Resultado de la auditoría
2. Palabras clave, anotaciones, librerías y métodos
3. Estructura del proyecto
4. Principales métodos y llamadas
5. Relaciones entre entidades
6. Funcionamiento de cada módulo
7. Ejemplos de ejecución paso a paso
8. Diagramas de relaciones y llamadas
9. Resumen para estudiar
10. Cierre y verificación
11. Validaciones definitivas de fechas en Solicitud y Servicio

1. Resultado de la auditoría

1.1 Archivos revisados

Se revisó el proyecto completo en estas áreas:

- **Java:** entidades, repositorios, servicios, controladores, configuración, seguridad, DTO de login/registro, validadores y clase principal.
- **Recursos:** todos los archivos HTML, todos los JavaScript y la hoja de estilos `style.css`.
- **Configuración:** `pom.xml`, `application.properties` y la prueba existente `SistemParcanApplicationTests.java`.

1.2 Comentarios nuevos agregados

Archivo	Documentación agregada
`src/main/resources/application.properties`	Explicación de `ddl-auto=update`, `show-sql`, `enable_lazy_load_no_trans` y mensajes/trazas de error.
`src/main/java/com/sistemparcan/controller/SesionController.java`	Documentación del parámetro `Authentication` y del valor devuelto.
`src/main/java/com/sistemparcan/service/UsuarioService.java`	Documentación del método que obtiene nombre e iniciales del funcionario.
`src/test/java/com/sistemparcan/SistemParcanApplicationTests.java`	Comentario sobre la prueba de carga del contexto.
`src/main/resources/static/js/app.js`	Comentario actualizado de `renderTable` para indicar el índice alfabético opcional.
`src/main/resources/static/js/funcionarios.js`	Comentario actualizado para indicar filtros y ordenamiento.
`src/main/java/com/sistemparcan/config/GlobalExceptionHandler.java`	Documentación del manejador que devuelve mensajes de validación al frontend.
`src/main/resources/static/js/app.js`	Funciones compartidas para mostrar errores encima de cada campo, resaltarlos y llevar al primero.
`src/main/resources/static/js/cargos.js`, `personas.js`, `usuarios.js`, `solicitudes.js`, `servicios.js`, `instituciones.js`, `roles.js`	Uso del sistema compartido de errores por campo.
`src/main/resources/static/css/style.css`	Estilos para el mensaje de error y el borde rojo del campo inválido.

1.3 Verificación realizada

> **Resultado:** se ejecutó `mvnw -q test` con Java 21 y la variable `DB_PASSWORD`. La prueba existente `contextLoads()` pasó correctamente. La compilación terminó sin errores.

No se modificó lógica, endpoints, consultas, relaciones, HTML funcional, CSS visual ni base de datos. No se ejecutaron pruebas funcionales nuevas de navegador porque los cambios de esta auditoría fueron únicamente comentarios.

1.4 Detalles que conviene revisar

1. **Thymeleaf está como dependencia**, pero no hay carpeta `templates/`. Las pantallas son HTML estático dentro de `static/`; Thymeleaf no se usa para generar vistas.
2. **Lombok está como dependencia**, pero las entidades tienen getters y setters manuales. No se encontraron anotaciones de Lombok.
3. **Roles existe como tabla y entidad**, pero el formulario de Usuarios no asigna roles y Spring Security asigna a todos los usuarios autenticados el rol fijo `USUARIO`.
4. **`Solicitud.funcionario` existe en la entidad**, pero el formulario de Solicitudes no envía un funcionario; normalmente queda en `null` desde la interfaz.
5. **Detalles de Solicitud y Servicio:** la validación permite 50 caracteres, pero la entidad/columna usa 45. Un texto de 46 a 50 podría fallar al guardar.
6. `PersonaService.cambiarCi` copia los registros a la nueva cédula y elimina físicamente la persona/funcionario antiguos. Es distinto de la eliminación lógica del resto del sistema.
7. `ddl-auto=update` y `enable_lazy_load_no_trans=true` funcionan, pero conviene conocer sus efectos: Hibernate puede ajustar el esquema y permite cargar relaciones lazy fuera de transacción.
8. **Solo existe la prueba** `contextLoads`; no hay pruebas automatizadas de reglas de negocio.

1.5 Sistema de errores por campo agregado

Se incorporó una mejora visual y de navegación para los formularios. Las reglas de validación existentes no se modificaron; ahora sus resultados se muestran junto al campo correspondiente.

- El mensaje aparece encima del campo inválido.
- El campo se marca con borde rojo.
- Se usa `aria-invalid` y `aria-describedby`.
- La pantalla se desplaza al primer campo inválido y coloca el foco.
- Al corregir un campo, su mensaje desaparece; los demás permanecen.
- Los errores generales, de conexión o de servidor conservan la alerta superior.
- Las validaciones del backend siguen siendo obligatorias.

Archivos principales: `app.js`, `cargos.js`, `personas.js`, `usuarios.js`, `solicitudes.js`, `servicios.js`, `instituciones.js`, `roles.js`, `style.css` y `GlobalExceptionHandler.java`.

2. Palabras clave, anotaciones, librerías y métodos

2.1 Palabras clave de Java

Elemento	¿Qué significa o para qué sirve?
`package`	Indica el paquete al que pertenece una clase.
`import`	Permite usar clases de otros paquetes.
`public`	Permite acceso desde otras clases.
`private`	Restringe el acceso al interior de la clase.
`class`	Define una clase.
`interface`	Define un contrato que otra clase implementa.
`extends`	Indica herencia o que una interfaz extiende otra.
`implements`	Indica que una clase implementa una interfaz.
`return`	Devuelve un valor desde un método.
`void`	Indica que un método no devuelve valor.
`new`	Crea un objeto nuevo.
`static`	Pertenece a la clase, no a una instancia.
`final`	Impide cambiar una referencia, clase o valor.
`boolean`	Tipo de dato verdadero/falso.
`int` / `Integer`	Número entero primitivo / objeto envoltorio.
`String`	Texto.
`null`	Ausencia de valor.
`if` / `else`	Toma de decisiones.
`for`	Repetición.
`try` / `catch`	Manejo de errores.
`this`	Referencia al objeto actual.
`@Override`	Indica que un método sobrescribe otro.

2.2 Anotaciones de Spring Boot y JPA

Anotación	¿Para qué se usa en SystemParCan?
`@SpringBootApplication`	Inicia la aplicación Spring Boot.
`@Entity`	Convierte una clase en tabla JPA.
`@Table`	Indica el nombre real de la tabla.
`@Id`	Marca la clave primaria.
`@GeneratedValue`	Genera automáticamente IDs numéricos.
`@Column`	Relaciona un atributo con una columna.
`@ManyToOne`	Muchos registros apuntan a uno.
`@OneToMany`	Un registro tiene muchos relacionados.
`@ManyToMany`	Relación muchos a muchos.
`@JoinColumn`	Define la clave foránea.
`@JoinTable`	Define una tabla intermedia.
`@CascadeType`	Indica operaciones que se propagan.
`@FetchType`	Define carga inmediata o perezosa.
`@Transient`	Campo que no se guarda en la base.
`@PrePersist`	Método que se ejecuta antes de insertar.

Anotación	¿Para qué se usa en SystemParCan?
`@JsonProperty`	Controla cómo se serializa un campo JSON.
`@JsonIgnore`	Oculta un campo en JSON.
`@JsonIgnoreProperties`	Evita ciclos al serializar relaciones.
`@JsonCreator`	Marca un constructor para deserialización JSON.
`@Service`	Declara una clase de lógica de negocio.
`@Repository`	Declara un repositorio de acceso a datos.
`@Controller`	Declara un controlador web.
`@RestController`	Controlador que devuelve JSON directamente.
`@RequestMapping`	Define la ruta base.
`@GetMapping`	Atiende peticiones GET.
`@PostMapping`	Atiende peticiones POST.
`@PutMapping`	Atiende peticiones PUT.
`@DeleteMapping`	Atiende peticiones DELETE.
`@PathVariable`	Toma un valor de la URL.
`@RequestBody`	Convierte el JSON recibido en objeto Java.
`@ResponseBody`	Devuelve el resultado como cuerpo de respuesta.
`@Configuration`	Declara configuración de Spring.
`@Bean`	Registra un objeto administrado por Spring.
`@Component`	Registra una clase como componente.
`@Transactional`	Ejecuta el método dentro de una transacción.
`@SpringBootTest`	Inicia el contexto para pruebas.
`@Test`	Marca un método como prueba JUnit.
`@RestControllerAdvice`	Agrupa manejadores globales de errores para los controladores REST.
`@ExceptionHandler`	Indica qué método atiende un tipo de excepción.

Nota: `@Autowired` no se usa en este proyecto; las dependencias se reciben por constructor.

2.3 Librerías y dependencias

Librería / dependencia	Para qué sirve	Dónde se usa
Spring Boot 4.1.1	Base del proyecto y autoconfiguración.	`pom.xml`, `SistemParcanApplication`.
Java 21	Lenguaje y versión de ejecución.	`pom.xml`, código Java.
Maven	Compila, prueba y empaqueta.	`pom.xml`, `mvnw`.
Spring Data JPA	Repositorios y acceso a datos.	Interfaces `*Repository`.
Hibernate	Implementación de JPA y generación de SQL.	Entidades y consultas.
Spring Web MVC	Controladores HTTP y API REST.	Clases `*Controller`.
Jackson	Conversión entre JSON y objetos Java.	Peticiones y respuestas de la API.
PostgreSQL Driver	Conexión con PostgreSQL.	`application.properties`.
Spring Security	Login, sesiones y protección de rutas.	`SecurityConfig`.
Spring Security Crypto	BCrypt y utilidades de contraseñas.	`PasswordService`.
JUnit 5 / Spring Boot Test	Pruebas automatizadas.	`SistemParcanApplicationTests`.
Thymeleaf	Motor de plantillas incluido, pero no utilizado actualmente.	Dependencia de `pom.xml`.
Lombok	Dependencia incluida, pero no se usan sus anotaciones.	`pom.xml`.
`java.time`	Fechas y horas.	`Persona`, `Solicitud`, `Servicio`.
`javax.crypto`	Huella HMAC-SHA256.	`PasswordService`.

2.4 Métodos de frameworks y librerías

Método	¿Qué hace?
`repository.findAll()`	Devuelve todos los registros.
`repository.findById(id)`	Busca por clave primaria.
`repository.save(entidad)`	Inserta o actualiza.
`repository.findBy...`	Consulta derivada del nombre del método.
`Optional.orElse()`	Devuelve un valor alternativo si no existe.
`Optional.ifPresent()`	Ejecuta código solo si hay valor.
`Optional.orElseThrow()`	Lanza error si no existe.
`ResponseEntity.ok()`	Respuesta HTTP 200 con cuerpo.
`ResponseEntity.notFound()`	Respuesta HTTP 404.
`Map.of()`	Crea un mapa inmutable.
`String.trim()`	Quita espacios de los extremos.
`String.split()`	Divide un texto.
`String.toUpperCase()`	Convierte a mayúsculas.
`String.matches()`	Valida con expresión regular.
`LocalDate.now()`	Fecha actual.
`BCryptPasswordEncoder.encode()`	Genera hash BCrypt.
`BCryptPasswordEncoder.matches()`	Compara contraseña con hash.
`Authentication.getName()`	Nombre del usuario autenticado.
`HttpSecurity`	Configura seguridad web.
`CommandLineRunner.run()`	Ejecuta código al iniciar Spring Boot.
`EntityManager`	Operaciones JPA avanzadas.

2.5 Métodos propios importantes

Clase	Método	Función
`PersonaService`	`registrar`	Valida y registra una persona; crea funcionario si corresponde.
`PersonaService`	`actualizar`	Actualiza persona y funcionario; puede cambiar la cédula.
`PersonaService`	`guardarFuncionarioSiCorresponde`	Crea o actualiza el funcionario asociado.
`PersonaService`	`cambiarCi`	Copia registros a la nueva cédula y actualiza referencias.
`PersonaService`	`validarFecha`	Valida fecha y edad mínima.
`PersonaService`	`validarDatosPersona`	Valida nombre, apellido, teléfono, dirección y cédula.
`PersonaService`	`deleteById` / `restoreById`	Eliminación lógica y restauración.
`FuncionarioService`	`findActivos`	Funcionarios activos con persona activa.
`FuncionarioService`	`restoreById`	Reactiva funcionario y persona.
`CargoService`	`save`	Valida y guarda un cargo.
`CargoService`	`findActivos`	Cargos activos para desplegables.
`UsuarioService`	`save`	Valida usuario, contraseña, funcionario, cargo y unicidad.
`UsuarioService`	`resolverCargo`	Resuelve el cargo real y controla inactivos.
`UsuarioService`	`obtenerSaludoFuncionario`	Obtiene nombre e iniciales del funcionario autenticado.
`UsuarioService`	`validarContrasena`	Valida requisitos de contraseña.
`SolicitudService`	`save`	Valida detalle y solicitante, y guarda la solicitud.
`SolicitudService`	`validarSolicitante`	Exige exactamente un feligrés o una institución.
`ServicioService`	`save`	Valida y guarda el servicio con sus relaciones.
`ServicioService`	`validarYResolverSolicitud`	Verifica la solicitud y su estado.

Clase	Método	Función
`ServicioService`	`validarYResolverFuncionario`	Verifica el funcionario y su persona.
`InstitucionEcclesiasticaService`	`findActivas`	Instituciones activas.
`RolesService`	`save`, `deleteById`, `restoreById`	CRUD y borrado lógico de roles.
`PasswordService`	`hash`, `verificar`, `huella`	Seguridad de contraseñas.
`LoginService`	`autenticar`	Verifica credenciales.
`SesionController`	`saludo`	Devuelve nombre e iniciales al frontend.
`SecurityConfig`	`userDetailsService`	Carga el usuario para Spring Security.
`SecurityConfig`	`securityFilterChain`	Define rutas públicas y protegidas.
`UsuarioSecurityMigration`	`run`	Completa datos de seguridad de usuarios antiguos.
`DataFixRunner`	`run`	Corrige fechas nulas de personas.
`GlobalExceptionHandler`	`manejarResponseStatus`	Devuelve el mensaje de negocio de las validaciones al frontend.

Los controladores de cada módulo repiten el patrón `findAll`, `findById`, `save`, `update`, `deleteById` y `restoreById`. `PersonaController` agrega `registrar` y `actualizar`; `UsuarioController` agrega validaciones de contraseña.

2.6 Funciones importantes de JavaScript

Archivo	Función	Qué hace
`app.js`	`fetchAll`	Hace peticiones y devuelve JSON.
`app.js`	`renderTable`	Construye las tablas.
`app.js`	`mostrarErrorCampo`	Muestra el mensaje encima del campo y lo marca en rojo.
`app.js`	`limpiarErrorCampo`	Retira el error de un campo corregido.
`app.js`	`mostrarErroresFormulario`	Muestra todos los errores y lleva al primero.
`app.js`	`erroresValidacionNativa`	Obtiene errores de las reglas HTML.
`app.js`	`revalidarCampo`	Actualiza o quita un error mientras se escribe.
`app.js`	`cargarSaludo`	Muestra nombre e iniciales.
`app.js`	`showAlert`	Muestra mensajes temporales.
`app.js`	`crearBuscadorSelect`	Buscador de desplegables.
`app.js`	`ordenarPorId`	Ordena por ID.
`app.js`	`ordenarAlfabeticamente`	Ordena por nombre, apellido e ID.
`app.js`	`letraInicial`	Obtiene la letra inicial.
`app.js`	`crearIndiceAlfabetico`	Dibuja el índice A-Z.
`personas.js`	`getPersonaRows`	Filtra personas por texto, estado y rol.
`personas.js`	`renderPersonas`	Ordena y dibuja la lista.
`funcionarios.js`	`renderFuncionarios`	Ordena y dibuja funcionarios.
`solicitudes.js`	`cargarSolicitantes`	Carga personas e instituciones activas.
`solicitudes.js`	`mostrarGrupoSolicitante`	Muestra el campo según el tipo.
`servicios.js`	`cargarSolicitudes`	Carga solicitudes activas.
`usuarios.js`	`cargarFuncionarios`	Carga funcionarios activos.
`usuarios.js`	`actualizarCargosFuncionario`	Ajusta cargos según funcionario.
`login.js`	`togglePassword`	Muestra u oculta contraseña.
`registro.js`	`validarContrasenaLocal`	Valida contraseña antes de enviar.

3. Estructura del proyecto

3.1 Capas

Capa	Función	Ejemplo real
Entity	Representa una tabla y sus columnas.	`Persona.java`, `Solicitud.java`.
Repository	Acceso a datos mediante Spring Data JPA.	`PersonaRepository`.
Service	Reglas de negocio y validaciones.	`PersonaService`, `UsuarioService`.
Controller	Recibe peticiones HTTP y devuelve respuestas.	`PersonaController`.
DTO	Transporta datos específicos.	`LoginRequest`, `LoginResponse`.
Config	Configuración de seguridad y datos.	`SecurityConfig`, `DataFixRunner`.
HTML	Estructura de las pantallas.	`personas.html`, `index.html`.
CSS	Apariencia visual.	`style.css`.
JavaScript	Eventos, validaciones y llamadas a la API.	`personas.js`, `app.js`.
PostgreSQL	Base de datos real.	`sistemparcan_bd`.

3.2 Comunicación entre capas

El flujo normal es:

```
HTML/JS → fetch → Controller → Service → Repository → Hibernate → PostgreSQL
```

Y la respuesta vuelve:

```
PostgreSQL → Repository → Service → Controller → JSON → JavaScript → tabla o mensaje
```

Ejemplo real:

`personas.js` envía `POST /api/personas/registro` → `PersonaController.registrar()` → `PersonaService.registrar()` → `PersonaRepository.save()` → Hibernate ejecuta el `INSERT` en PostgreSQL.

3.3 Por qué están separadas

- **Entity** solo describe datos.
- **Repository** solo consulta y guarda.
- **Service** aplica reglas, como evitar duplicados o validar edades.
- **Controller** atiende HTTP y no debería contener toda la lógica.
- **JavaScript** maneja la interfaz y la comunicación.
- **PostgreSQL** conserva la información.

Esta separación permite cambiar una pantalla sin reescribir la base y cambiar una regla sin tocar el HTML.

4. Principales métodos y llamadas

Método	Clase	Quién lo llama	Parámetros	Devuelve	Efecto en base
`registrar(Persona)`	`PersonaService`	`PersonaController.registrar`	Persona con rol/cargo	Persona guardada	Inserta persona y quizá funcionario.
`actualizar(Integer, Persona)`	`PersonaService`	`PersonaController.actualizar`	CI original y datos	Persona actualizada	Actualiza persona/funcionario.
`guardarFuncionarioSiCorresponde`	`PersonaService`	`registrar` / `actualizar`	Persona	Nada	Inserta o actualiza funcionario.
`save(Usuario)`	`UsuarioService`	`UsuarioController`	Usuario	Usuario guardado	Inserta/actualiza usuario.
`obtenerSaludoFuncionario`	`UsuarioService`	`SesionController.saludo`	Nombre de sesión	Map con nombre/iniciales	Solo lectura.
`save(Solicitud)`	`SolicitudService`	`SolicitudController`	Solicitud	Solicitud guardada	Inserta/actualiza solicitud.
`save(Servicio)`	`ServicioService`	`ServicioController`	Servicio	Servicio guardado	Inserta/actualiza servicio.
`deleteById(Integer)`	Servicios de módulos	Controladores	ID	Nada	Cambia `eliminado=true`.
`restoreById(Integer)`	Servicios de módulos	Controladores	ID	Nada	Cambia `eliminado=false`.
`autenticar(LoginRequest)`	`LoginService`	`LoginController`	Usuario/contraseña	`LoginResponse`	Solo lectura.
`saludo(Authentication)`	`SesionController`	`app.js`	Sesión	JSON nombre/iniciales	Solo lectura.

Las llamadas a `repository.save()` son las que realmente insertan o actualizan en PostgreSQL.

5. Relaciones entre entidades

Relación	Tipo JPA / clave	Clase donde está	Comportamiento real
Persona → Funcionario	`@ManyToOne`, `Persona_ciPersona`	`Funcionario.persona`	Un funcionario pertenece a una persona. Una persona puede no ser funcionario.
Cargo → Funcionario	`@ManyToOne`, `Cargo_codCargo`	`Funcionario.cargo`	Un funcionario puede tener un cargo. Un cargo puede estar en varios funcionarios.
Funcionario → Usuario	`@ManyToOne`, `Funcionario_ciPersona`	`Usuario.funcionario`	Un usuario puede estar asociado a un funcionario.
Cargo → Usuario	`@ManyToOne`, `Cargo_codCargo`	`Usuario.cargo`	Un usuario puede tener un cargo.
Usuario ↔ Roles	`@ManyToMany`, tabla `Funcionario_Roles`	`Usuario.roles` / `Roles.usuarios`	La relación existe en las entidades, pero la interfaz de Usuarios no asigna roles.
Persona → Solicitud	`@ManyToOne`, `Feligres_ciPersona`	`Solicitud.feligres`	Una solicitud puede tener un feligrés.
Institución → Solicitud	`@ManyToOne`, `Institucion_idInstitucionEclesiastica`	`Solicitud.institucion`	Una solicitud puede tener una institución.
Funcionario → Solicitud	`@ManyToOne`, `Funcionario_ciPersona`	`Solicitud.funcionario`	El campo existe, pero el formulario de Solicitudes no lo asigna.
Solicitud → Servicio	`@OneToMany`, `Solicitud_codSolicitud`	`Solicitud.servicios` / `Servicio.solicitud`	Una solicitud puede tener varios servicios.
Funcionario → Servicio	`@ManyToOne`, `Funcionario_ciPersona`	`Servicio.funcionario`	Un servicio tiene un funcionario responsable.
Funcionario → Servicios/Solicitudes	`@OneToMany`	`Funcionario.servicios` / `Funcionario.solicitudes`	Son colecciones inversas; no se borran físicamente al desactivar.

5.1 Comportamientos importantes

- Una **Persona** no tiene una colección de funcionarios. La relación se consulta desde `Funcionario.persona`.
- `PersonaService.deleteById()` solo marca `Persona.eliminado=true`. No desactiva automáticamente el funcionario.
- `FuncionarioService.deleteById()` solo marca `Funcionario.eliminado=true`. No desactiva la persona.
- Al **restaurar**, `PersonaService.restoreById()` reactiva también el funcionario si existe, y `FuncionarioService.restoreById()` reactiva también la persona.
- Al desactivar una persona, sus solicitudes históricas no se modifican.
- Al desactivar una institución, las solicitudes históricas tampoco se modifican.
- Al editar una solicitud o servicio antiguo, se permite conservar una relación inactiva que ya existía; no se permite asignar una relación inactiva nueva.
- `SolicitudService` exige exactamente un solicitante: feligrés o institución, nunca ambos.
- `ServicioService` exige solicitud y funcionario.
- `PersonaService.cambiarCi()` copia persona/funcionario a la nueva cédula, actualiza referencias en solicitud, servicio y usuario, y elimina los registros antiguos.
- Si se cambia el rol de funcionario a persona en el formulario, el funcionario existente no se elimina automáticamente.

6. Funcionamiento de cada módulo

6.1 Personas

Archivos: `personas.html`, `personas.js`, `PersonaController`, `PersonaService`, `PersonaRepository`, `Persona`.

- **Listar:** `loadPersonas()` consulta `/api/personas` y `/api/funcionarios`; `getPersonaRows()` filtra por texto, estado y rol; `renderPersonas()` ordena A-Z y dibuja la tabla.
- **Registrar:** formulario → `POST /api/personas/registro` → `PersonaService.registrar()`.
- **Editar:** `PUT /api/personas/registro/{ci}` → `PersonaService.actualizar()`.
- **Desactivar:** `DELETE /api/personas/{ci}` → `deleteById()`.
- **Restaurar:** `POST /api/personas/{ci}/restore` → `restoreById()`.
- **Roles:** el formulario permite PERSONA o FUNCIONARIO. Si es FUNCIONARIO, exige cargo y crea/actualiza el funcionario.
- **Orden e índice:** selector A-Z/Z-A e índice alfabético lateral.

6.2 Funcionarios

Archivos: `funcionarios.html`, `funcionarios.js`, `FuncionarioController`, `FuncionarioService`, `FuncionarioRepository`, `Funcionario`.

- **Listar:** `/api/funcionarios`, filtro por texto y estado.
- **Registrar/editar:** la interfaz principal se hace desde Personas; el botón Editar lleva a `personas.html`.
- **Desactivar/restaurar:** endpoints propios del controlador.
- **Orden e índice:** igual que Personas.
- **Relación:** cada funcionario apunta a una persona y a un cargo.
- **Restauración:** reactiva también la persona asociada.

6.3 Cargos

Archivos: `cargos.html`, `cargos.js`, `CargoController`, `CargoService`, `CargoRepository`, `Cargo`.

- CRUD completo con `POST`, `PUT`, `DELETE` y `restore`.
- `CargoService.save()` valida descripción, longitud y caracteres.
- `/api/cargos/activos` alimenta los desplegables de Personas, Usuarios y Registro.
- Desactivar un cargo no elimina las relaciones históricas de funcionarios/usuarios.

6.4 Usuarios

Archivos: `usuarios.html`, `usuarios.js`, `UsuarioController`, `UsuarioService`, `UsuarioRepository`, `Usuario`.

- **Listar:** `/api/usuarios`; se muestran funcionario y cargo separados.
- **Registrar:** `POST /api/usuarios`; se valida la contraseña y se guarda.
- **Editar:** `PUT /api/usuarios/{id}`; si no se envía contraseña, se conserva la actual.
- **Desactivar/restaurar:** borrado lógico.
- **Funcionario:** se elige de funcionarios activos.
- **Cargo:** se cargan cargos activos y se restringen según el funcionario.
- **Seguridad:** contraseña BCrypt, huella HMAC y validación de reutilización.
- **Relaciones:** usuario → funcionario y usuario → cargo.
- **Saludo:** `SesionController` usa la relación Usuario → Funcionario → Persona.

6.5 Roles

Archivos: `roles.html`, `roles.js`, `RolesController`, `RolesService`, `RolesRepository`, `Roles`.

- CRUD de roles con eliminación lógica y restauración.
- La entidad `Usuario` tiene una relación `@ManyToMany` con `Roles` mediante `Funcionario_Roles`.
- Actualmente el formulario de Usuarios no permite seleccionar roles.
- `SecurityConfig` no usa los roles de la tabla; asigna el rol fijo `USUARIO` a toda cuenta autenticada.
- Por eso todos los usuarios autenticados pueden entrar a los módulos protegidos.

6.6 Instituciones Eclesiásticas

Archivos: `instituciones.html`, `instituciones.js`, `InstitucionEcclesiasticaController`, `InstitucionEcclesiasticaService`, `InstitucionEcclesiasticaRepository`, `InstitucionEcclesiastica`.

- CRUD con borrado lógico.
- `/api/instituciones/activas` alimenta el desplegable de Solicitudes.
- Al crear una solicitud se puede elegir una institución en lugar de un feligrés.
- Las solicitudes históricas conservan la institución aunque luego se desactive.

6.7 Solicitudes

Archivos: `solicitudes.html`, `solicitudes.js`, `SolicitudController`, `SolicitudService`, `SolicitudRepository`, `Solicitud`.

- **Listar:** con filtros de texto y estado; muestra si está aprobada o pendiente y si está activa/inactiva.
- **Registrar:** el formulario elige tipo de solicitante: FELIGRÉS o INSTITUCIÓN.
- **Validación:** `SolicitudService.save()` exige un detalle y exactamente un solicitante.
- **Relaciones:** solicitud → persona (feligrés) o solicitud → institución.
- **Funcionario:** el campo existe en la entidad, pero el formulario no lo asigna.
- **Servicios:** una solicitud puede tener varios servicios mediante `Solicitud.servicios`.
- **Desactivar/restaurar:** solo cambia `eliminado`; no borra servicios relacionados.

6.8 Servicios

Archivos: `servicios.html`, `servicios.js`, `ServicioController`, `ServicioService`, `ServicioRepository`, `Servicio`.

- **Listar:** filtros por texto y estado realizado/pendiente.
- **Registrar:** requiere solicitud y funcionario.
- **Validaciones:** detalle, costo no negativo y estados activos.
- **Relaciones:** servicio → solicitud y servicio → funcionario.
- **Costo:** se escribe normalmente y se muestra con formato `1.000 Gs.`.
- **Desactivar/restaurar:** solo cambia `eliminado`; no modifica la solicitud ni el funcionario.

6.9 Login, autenticación y saludo

Archivos: `SecurityConfig`, `LoginController`, `LoginService`, `LoginRequest`, `LoginResponse`, `RegistroController`, `RegistroRequest`, `SesionController`, `login.html`, `login.js`, `registro.html`, `registro.js`, `app.js`.

- `SecurityConfig` protege las rutas y configura `/login`.
- `UserDetailsService` busca el usuario por `usuario_normalizado`.
- `PasswordEncoder` usa `PasswordService` para BCrypt.
- Si el usuario está eliminado, Spring Security lo marca como deshabilitado.
- `LoginController` ofrece además `/api/login`.
- `RegistroController` permite crear cuenta solo a un funcionario activo con cargo activo.
- `SesionController` obtiene el funcionario del usuario autenticado y devuelve nombre e iniciales.
- `app.js` muestra el saludo y el círculo con iniciales.
- Las rutas públicas son login, registro, CSS, JS, imágenes, `/api/login`, `/api/registro`, `/error` y `GET /api/cargos`.
- Las demás rutas requieren sesión autenticada. No hay autorización por roles específicos.

6.10 Sistema compartido de errores de formulario

Los formularios usan funciones compartidas de `app.js` para presentar los errores. Cada campo puede tener un mensaje encima, borde rojo y atributos de accesibilidad.

- **Validación nativa:** `erroresValidacionNativa(form)` revisa `required`, `pattern`, `minlength`, `maxlength`, fechas y otras restricciones HTML.
- **Validaciones propias:** cada módulo conserva sus funciones de validación y mensajes.
- **Visualización:** `mostrarErroresFormulario(form, errores)` muestra todos los mensajes, resalta los campos y lleva al primero.
- **Corrección:** `revalidarCampo(campo)` actualiza o elimina el error mientras el usuario escribe.
- **Accesibilidad:** se usan `aria-invalid`, `aria-describedby` y `role="alert"`.
- **Errores del servidor:** `GlobalExceptionHandler` devuelve el mensaje de negocio para asociarlo a un campo cuando sea posible.
- **Errores generales:** si no se puede asociar a un campo, se mantiene la alerta superior.

7. Ejemplos de ejecución paso a paso

7.1 Ejemplo A: registrar una persona

1. El usuario completa `personas.html` y presiona **Guardar**.
2. `personas.js` valida cédula, nombre, apellido, fecha, teléfono y dirección.
3. Envía `POST /api/personas/registro` con los datos.
4. `PersonaController.registrar()` recibe el JSON y llama a `PersonaService.registrar()`.
5. `registrar()` valida fecha y datos, revisa que la cédula no exista y asigna `fechaRegistro`.
6. Guarda la persona con `PersonaRepository.save()`.
7. Si el rol es FUNCIONARIO, `guardarFuncionarioSiCorresponde()` crea o actualiza el funcionario y le asigna persona y cargo.
8. Si el rol es PERSONA, no se crea funcionario.
9. La respuesta vuelve al navegador y `loadPersonas()` actualiza la tabla.

7.2 Ejemplo B: registrar una solicitud

1. El usuario elige **Feligrés** o **Institución**.
2. `mostrarGrupoSolicitante()` muestra el desplegable correspondiente.
3. `solicitudes.js` envía `POST /api/solicitudes` con `feligres` o `institucion`.
4. `SolicitudController.save()` llama a `SolicitudService.save()`.
5. `save()` valida detalle y solicitante.
6. Busca la persona o institución real en su repositorio.
7. Si está inactiva y no era la relación histórica, lanza error.
8. Asigna la relación y limpia la relación que no corresponde.
9. `SolicitudRepository.save()` guarda la solicitud.

7.3 Ejemplo C: registrar un servicio

1. `servicios.js` carga solicitudes y funcionarios activos.
2. El usuario elige ambos y completa detalle, costo, estado y fecha.
3. Envía `POST /api/servicios`.
4. `ServicioController.save()` llama a `ServicioService.save()`.
5. `save()` valida detalle y costo.
6. Busca la solicitud y el funcionario reales.
7. Verifica que estén activos; si el servicio se edita, permite conservar la relación inactiva que ya tenía.
8. Guarda el servicio con las claves foráneas correspondientes.

7.4 Ejemplo D: iniciar sesión y mostrar el saludo

1. El usuario envía el formulario de `login.html` a `/login`.
2. Spring Security usa `SecurityConfig.userDetailsService`.
3. `userDetailsService` normaliza el nombre y consulta `UsuarioRepository.findByUsuarioNormalizado`.
4. `PasswordEncoder.matches()` compara la contraseña con el hash.
5. Si es correcta, se crea la sesión y se redirige a `index.html`.
6. `app.js` ejecuta `cargarSaludo()` y llama a `/api/sesion/saludo`.
7. `SesionController.saludo()` recibe `Authentication`.
8. `UsuarioService.obtenerSaludoFuncionario()` busca usuario → funcionario → persona.
9. Devuelve el primer nombre y las iniciales.
10. `app.js` muestra, por ejemplo, **Hola, Alba** y el círculo **AF**.

7.5 Ejemplo E: desactivar y restaurar

1. El usuario presiona **Desactivar**.
2. JavaScript llama a `DELETE /api/.../{id}`.
3. El controller llama a `deleteById(id)`.
4. El service busca el registro y cambia `eliminado` a `true`.

5. No se borra la fila ni se eliminan las relaciones.
6. Las consultas de activos dejan de mostrarlo; las consultas históricas todavía pueden encontrarlo.
7. Al presionar **Activar**, se llama a `POST /api/.../{id}/restore`.
8. El service cambia `eliminado` a `false`.
9. En Personas y Funcionarios, la restauración sincroniza persona y funcionario.
10. En los demás módulos, solo se restaura el registro propio.

7.6 Ejemplo F: corregir errores de un formulario

1. El usuario presiona **Guardar** con datos incorrectos.
2. El navegador ejecuta el manejador del formulario y reúne las reglas HTML y las reglas propias del módulo.
3. `mostrarErroresFormulario()` coloca un mensaje encima de cada campo inválido.
4. Cada campo recibe el borde rojo y `aria-invalid="true"`.
5. La pantalla se desplaza al primer campo inválido y coloca el foco.
6. Mientras el usuario escribe, `revalidarCampo()` revisa el campo y retira solo su error si ya es válido.
7. Los datos de los demás campos se conservan.
8. Si el backend rechaza el formulario, `GlobalExceptionHandler` devuelve el mensaje y el frontend lo asocia al campo correspondiente cuando es posible.
9. Si el error es general, se mantiene la alerta superior.

8. Diagramas de relaciones y llamadas

8.1 Diagrama de relaciones entre entidades

```
PERSONA 1 ──────── 0..1 FUNCIONARIO
CARGO 1 ─────────── * FUNCIONARIO
FUNCIONARIO 1 ─ * USUARIO
CARGO 1 ─────────── * USUARIO
USUARIO * ──────── * ROLES          (tabla Funcionario_Roles)

PERSONA 1 ──────── * SOLICITUD      (feligrés)
INSTITUCION 1 ─ * SOLICITUD      (institución)
FUNCIONARIO 1 ─ * SOLICITUD      (campo opcional)
SOLICITUD 1 ─── * SERVICIO
FUNCIONARIO 1 ─ * SERVICIO       (responsable)
```

La relación Persona–Funcionario se declara desde `Funcionario.persona`. Persona no tiene una colección inversa de funcionarios.

8.2 Diagrama de flujo de llamadas

```
NAVEGADOR
  HTML + CSS + JavaScript
        │
        │ fetch(JSON)
        ▼
CONTROLLER  ──────────────►  SERVICE
        ▲                       │
        │                       ▼
        │                  REPOSITORY
        │                       │
        │                       ▼
        │                   HIBERNATE
        │                       │
        │                       ▼
        └──────────────────  POSTGRESQL

LOGIN:
Formulario login → SecurityConfig → UserDetailsService
                 → UsuarioRepository → PasswordService

SALUDO:
app.js → GET /api/sesion/saludo → SesionController
       → UsuarioService → UsuarioRepository → Funcionario → Persona
```

9. Resumen para estudiar

9.1 Glosario

Concepto	Explicación sencilla
Entidad	Clase que representa una tabla.
Repository	Interfaz que consulta y guarda datos.
Service	Clase donde están las reglas del negocio.
Controller	Recibe las peticiones web.
DTO	Objeto para transportar datos de entrada/salida.
JPA	Estándar para mapear objetos a tablas.
Hibernate	Implementación de JPA que genera SQL.
Spring Boot	Framework que configura y arranca la aplicación.
Inyección de dependencias	Spring entrega los objetos que una clase necesita.
Transacción	Grupo de operaciones que se confirman o cancelan juntas.
Eliminación lógica	Marcar `eliminado=true` en lugar de borrar la fila.
Clave foránea	Columna que apunta al ID de otra tabla.
Lazy loading	Cargar una relación solo cuando se necesita.
BCrypt	Algoritmo para guardar contraseñas con hash.
HMAC	Huella para detectar contraseñas repetidas sin guardarlas en claro.
Endpoint	Ruta de la API, por ejemplo `/api/personas`.
JSON	Formato de texto para intercambiar datos.

9.2 Los 15 métodos o funciones más importantes

1. `PersonaService.registrar()`: registra personas y crea funcionarios.
2. `PersonaService.actualizar()`: actualiza personas y funcionarios.
3. `PersonaService.guardarFuncionarioSiCorresponde()`: crea/actualiza funcionario.
4. `PersonaService.cambiarCi()`: cambia cédula y referencias.
5. `UsuarioService.save()`: guarda usuarios y valida relaciones.
6. `UsuarioService.obtenerSaludoFuncionario()`: obtiene nombre e iniciales.
7. `SolicitudService.save()`: guarda solicitudes y su solicitante.
8. `ServicioService.save()`: guarda servicios y sus relaciones.
9. `PasswordService.hash()`: genera hash BCrypt.
10. `PasswordService.verificar()`: compara contraseña.
11. `SecurityConfig.userDetailsService()`: carga el usuario para login.
12. `LoginService.autenticar()`: valida credenciales.
13. `SesionController.saludo()`: entrega el saludo al frontend.
14. `app.js cargarSaludo()`: muestra nombre e iniciales.
15. `app.js renderTable()`: construye las tablas y el índice alfabético.

9.3 Relaciones principales entre módulos

- **Personas** alimenta los datos de **Funcionarios**.
- **Funcionarios** usan **Cargos**.
- **Usuarios** se relacionan con **Funcionarios** y **Cargos**.
- **Usuarios** tienen una relación ManyToMany con **Roles**, aunque la interfaz todavía no la usa.
- **Solicitudes** se relacionan con **Personas** o **Instituciones**.
- **Servicios** se relacionan con **Solicitudes** y **Funcionarios**.
- **Login/Security** controla el acceso a todos los módulos.

9.4 Diferencias entre Entity, Repository, Service y Controller

Capa	Pregunta que responde
Entity	¿Cómo se guarda este dato en la tabla?
Repository	¿Cómo busco y guardo los datos?
Service	¿Qué reglas debo cumplir antes de guardar?
Controller	¿Qué ruta HTTP recibe la petición y qué devuelvo?

9.5 Recorrido recomendado para estudiar

1. `SistemParcanApplication.java`.
2. Una entidad sencilla: `Cargo.java`.
3. Su repositorio: `CargoRepository.java`.
4. Su servicio: `CargoService.java`.
5. Su controlador: `CargoController.java`.
6. Su pantalla: `cargos.html` y `cargos.js`.
7. Luego `Persona`, `Funcionario`, `Usuario`.
8. Después `Solicitud` y `Servicio`.
9. Finalmente `SecurityConfig`, `PasswordService`, `LoginService` y `SesionController`.
10. Para terminar, `app.js`, porque concentra las funciones compartidas del frontend.

9.6 Funciones del sistema de errores por campo

Función / método	Archivo	Función
`mostrarErrorCampo`	`app.js`	Crea el mensaje encima del campo y lo marca en rojo.
`limpiarErrorCampo`	`app.js`	Retira el error y los atributos de accesibilidad.
`limpiarErroresFormulario`	`app.js`	Limpia todos los errores de un formulario.
`mostrarErroresFormulario`	`app.js`	Muestra todos los errores y lleva al primero.
`mensajeValidacionNativa`	`app.js`	Traduce las restricciones HTML a mensajes.
`erroresValidacionNativa`	`app.js`	Recoge los errores de las reglas HTML.
`revalidarCampo`	`app.js`	Actualiza o limpia el error mientras se escribe.
`manejarResponseStatus`	`GlobalExceptionHandler`	Devuelve el mensaje de negocio del backend.

10. Cierre y verificación

10.1 Archivos que recibieron comentarios nuevos

- `src/main/resources/application.properties`
- `src/main/java/com/sistemparcan/controller/SesionController.java`
- `src/main/java/com/sistemparcan/service/UsuarioService.java`
- `src/test/java/com/sistemparcan/SistemParcanApplicationTests.java`
- `src/main/resources/static/js/app.js`
- `src/main/resources/static/js/funcionarios.js`
- `src/main/java/com/sistemparcan/config/GlobalExceptionHandler.java`
- `src/main/resources/static/js/cargos.js`
- `src/main/resources/static/js/personas.js`
- `src/main/resources/static/js/usuarios.js`
- `src/main/resources/static/js/solicitudes.js`
- `src/main/resources/static/js/servicios.js`
- `src/main/resources/static/js/instituciones.js`
- `src/main/resources/static/js/roles.js`
- `src/main/resources/static/css/style.css`

10.2 Confirmación

- No se modificó la lógica de negocio ni las reglas de validación.
- Se agregó la presentación de errores por campo y el manejador de mensajes del backend.
- No se cambiaron endpoints, consultas, relaciones ni estructura de la base de datos.
- No se alteró la distribución visual general.
- Se ejecutó `mvnw -q test`; la prueba existente de carga de contexto pasó.
- Se observaron advertencias de Hibernate/Spring sobre el dialecto PostgreSQL, `open-in-view`, ausencia de plantillas Thymeleaf y Mockito; no impidieron la ejecución.

10.3 Pruebas realizadas para los errores por campo

- Formulario vacío de Personas: errores en CI, Nombre, Apellido y Fecha, con foco en el primer campo.
- Varios errores simultáneos en Personas: cédula inválida, teléfono con letras y dirección con espacios.
- Corrección de un solo campo: se retiró únicamente el error de ese campo.
- Validación de cargo mientras se escribe: el mensaje cambió y desapareció al corregir.
- Usuarios: errores de usuario y contraseña, con limpieza independiente.
- Solicitudes: error de tipo de solicitante, detalle y feligrés.
- Servicios: errores de solicitud, detalle y funcionario.
- Fecha futura en Personas: error y foco en el campo de fecha.
- Edición de una persona con teléfono inválido.
- Errores del backend por cédula duplicada y usuario duplicado, mostrados junto al campo.
- Error general simulado: se conservó la alerta superior.
- Navegación por todos los módulos sin errores de JavaScript en consola.

> **Resumen final:** SystemParCan está organizado por capas, usa Spring Boot + Spring Data JPA + Hibernate para acceder a PostgreSQL, Spring Security para el login, y JavaScript con Fetch API para comunicar las pantallas con los controladores. La eliminación es lógica, las relaciones se manejan con claves foráneas JPA y las operaciones principales pasan por Controller → Service → Repository → Base de datos. Los formularios ahora muestran los errores junto a cada campo, con desplazamiento al primer error, foco, borde rojo y limpieza automática al corregir.

11. Validaciones definitivas de fechas en Solicitud y Servicio

11.1 Introducción

Esta actualización agrega dos reglas de negocio para proteger las fechas de Solicitud y Servicio. La validación se aplica tanto en el navegador como en el backend, reutilizando el sistema visual de errores por campo. No se crearon clases DTO ni se modificó el esquema de PostgreSQL.

11.2 Reglas implementadas

Regla	Condición
Fecha de solicitud	Debe estar entre la fecha actual menos seis meses calendario y la fecha actual, incluyendo ambos extremos.
Fecha de realización	Debe ser mayor o igual a la fecha de la solicitud asociada y menor o igual a la fecha actual.
Edición de solicitud	La nueva fecha no puede ser posterior a la fecha de realización de ningún servicio asociado.

11.3 Relación entre Solicitud y Servicio

La relación real está definida en `Servicio.solicitud` como `@ManyToOne` con la clave `Solicitud_codSolicitud`. Una solicitud puede tener varios servicios. La fecha de solicitud se obtiene desde la entidad `Solicitud` cargada por el repositorio, no desde datos enviados por el navegador.

Para consultar los servicios asociados se agregó en `ServicioRepository`:

```
List<Servicio> findBySolicitudCodSolicitud(Integer codSolicitud);
```

11.4 Clases y archivos modificados

Archivo	Cambio
`ServicioRepository.java`	Consulta de servicios por solicitud.
`SolicitudService.java`	Validación de fecha de solicitud y de servicios asociados.
`ServicioService.java`	Validación de fecha de realización contra la solicitud y la fecha actual.
`GlobalExceptionHandler.java`	Mensaje para fechas con formato incorrecto.
`Funcionario.java`	Constructor marcado con `@JsonCreator` para deserializar correctamente el funcionario del servicio.
`app.js`	Funciones de fecha calendario, validación de fecha ISO y límites.
`solicitudes.js`	Validación visual de la fecha de solicitud.
`servicios.js`	Límites dinámicos según la solicitud y validación de la fecha de realización.

11.5 Validación de fecha de solicitud

En `SolicitudService.save()` se llama a `validarFechaSolicitud()` antes de guardar. El método usa la zona horaria local del equipo:

```
LocalDate hoy = LocalDate.now(ZoneId.systemDefault());
LocalDate fechaMinima = hoy.minusMonths(6);

if (fechaSolicitud == null) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
        "Fecha inválida. Debe ingresar una fecha de solicitud.");
}
if (fechaSolicitud.isBefore(fechaMinima)) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
        "Fecha inválida. Ingrese otra fecha. La fecha de solicitud no puede ser anterior a los últimos seis meses.");
}
if (fechaSolicitud.isAfter(hoy)) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
        "Fecha inválida. Ingrese otra fecha. La fecha de solicitud no puede ser posterior a la fecha actual.");
}
```

`LocalDate.minusMonths(6)` respeta meses calendario, fines de mes y años bisiestos; no usa una aproximación de 180 días.

11.6 Validación de fecha de realización

En `ServicioService.save()`, después de resolver la solicitud real, se ejecuta `validarFechaRealizacion()`:

```
LocalDate fechaRealizacion = servicio.getFechaRealizacion();
if (fechaRealizacion == null) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
        "Fecha inválida. Debe ingresar una fecha de realización.");
}

Solicitud solicitud = servicio.getSolicitud();
if (solicitud == null || solicitud.getFechaSolicitud() == null) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
        "No se puede validar la fecha de realización. Seleccione primero una solicitud válida.");
}

LocalDate hoy = LocalDate.now(ZoneId.systemDefault());
if (fechaRealizacion.isBefore(solicitud.getFechaSolicitud())) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
        "Fecha inválida. Ingrese otra fecha. La fecha de realización no puede ser anterior a la fecha de solicitud.");
}
if (fechaRealizacion.isAfter(hoy)) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
        "Fecha inválida. Ingrese otra fecha. La fecha de realización no puede ser posterior a la fecha actual.");
}
```

11.7 Edición de solicitudes con servicios asociados

Al editar una solicitud, `SolicitudService` consulta sus servicios mediante `servicioRepository.findBySolicitudCodSolicitud()`. Si la nueva fecha de solicitud es posterior a la fecha de realización de algún servicio, se rechaza el cambio:

```
for (Servicio servicio : servicioRepository.findBySolicitudCodSolicitud(codSolicitud)) {
    if (servicio.getFechaRealizacion() != null
            && fechaSolicitud.isAfter(servicio.getFechaRealizacion())) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
            "Fecha inválida. Ingrese otra fecha. La fecha de solicitud no puede ser posterior a la fecha de realización de un servicio
asociado.");
    }
}
```

11.8 Validaciones del frontend

`app.js` incorpora:

- `fechaISO()`: convierte una fecha local a `YYYY-MM-DD`.
- `fechaHaceMeses(6)`: calcula seis meses calendario y ajusta el día al último del mes cuando corresponde.
- `esFechaISOValida()`: comprueba que la fecha exista realmente.

`solicitudes.js` configura `required`, `min` y `max` del campo, y valida con los mismos mensajes del backend. `servicios.js` guarda la fecha de cada solicitud en `fechasSolicitud`, actualiza `min` y `max` al cambiar la solicitud y muestra inmediatamente el error si la fecha ya ingresada deja de ser válida.

11.9 Backend sin DTO y zona horaria

No se crearon clases DTO. Se reutilizaron las entidades `Solicitud` y `Servicio`, los controladores existentes, los servicios y los repositorios. La fecha actual se obtiene con `LocalDate.now(ZoneId.systemDefault())`, usando la zona horaria local configurada en el equipo de desarrollo.

`GlobalExceptionHandler` también devuelve un mensaje de fecha cuando el JSON contiene una fecha inexistente o con formato incorrecto, para que el frontend pueda mostrarlo junto al campo.

11.10 Mensajes y condiciones

Situación	Mensaje
Solicitud sin fecha	Fecha inválida. Debe ingresar una fecha de solicitud.
Solicitud antes de seis meses	Fecha inválida. Ingrese otra fecha. La fecha de solicitud no puede ser anterior a los últimos seis meses.
Solicitud futura	Fecha inválida. Ingrese otra fecha. La fecha de solicitud no puede ser posterior a la fecha actual.
Servicio sin fecha	Fecha inválida. Debe ingresar una fecha de realización.
Servicio sin solicitud	No se puede validar la fecha de realización. Seleccione primero una solicitud válida.
Realización anterior a la solicitud	Fecha inválida. Ingrese otra fecha. La fecha de realización no puede ser anterior a la fecha de solicitud.
Realización futura	Fecha inválida. Ingrese otra fecha. La fecha de realización no puede ser posterior a la fecha actual.
Solicitud posterior a un servicio asociado	Fecha inválida. Ingrese otra fecha. La fecha de solicitud no puede ser posterior a la fecha de realización de un servicio asociado.

Situación	Mensaje
Fecha inexistente o formato incorrecto	Fecha inválida. Ingrese una fecha de solicitud válida. / Fecha inválida. Ingrese una fecha de realización válida.

11.11 Ejemplos de fechas

Fecha de solicitud	Resultado con hoy 12/11/2026
2026-05-11	Inválida: anterior a seis meses.
2026-05-12	Válida: límite mínimo exacto.
2026-08-15	Válida.
2026-11-12	Válida: fecha actual.
2026-11-13	Inválida: futura.

Fecha de realización	Resultado con solicitud 2026-11-05 y hoy 2026-11-12
2026-11-04	Inválida: anterior a la solicitud.
2026-11-05	Válida: igual a la solicitud.
2026-11-08	Válida.
2026-11-12	Válida: fecha actual.
2026-11-13	Inválida: futura.

11.12 Funcionamiento paso a paso

1. El usuario selecciona una solicitud en Servicios.
2. `servicios.js` obtiene su fecha desde `fechasSolicitud` y actualiza `min` y `max` del campo de realización.
3. Al guardar, el frontend valida la fecha y muestra el error encima del campo si corresponde.
4. El navegador envía la entidad Servicio al controlador.
5. `ServicioService` consulta la solicitud real mediante el repositorio.
6. Comprueba que la fecha de realización esté entre la fecha de la solicitud y hoy.
7. Si es válida, guarda; si no, devuelve el mensaje de negocio y no modifica la base.

11.13 Pruebas realizadas

Prueba	Resultado
Solicitud con fecha 2026-05-11	400, mensaje de fecha anterior al límite.
Solicitud con fecha 2026-11-13	400, mensaje de fecha futura.
Solicitud sin fecha	400, mensaje de fecha obligatoria.
Solicitud con 2026-05-12	200, registro temporal creado y eliminado después.
Solicitud con 2026-11-12	200, registro temporal creado y eliminado después.
Fecha inexistente 2026-02-30	400 con mensaje de fecha inválida.
Servicio sin solicitud	400 con mensaje de seleccionar solicitud.
Servicio con fecha anterior a la solicitud	400 con mensaje correspondiente.
Servicio con fecha futura	400 con mensaje correspondiente.
Servicio sin fecha	400 con mensaje de fecha obligatoria.
Servicio con fecha igual a la solicitud	200.
Servicio con fecha posterior y anterior a hoy	200.
Servicio con fecha igual a hoy	200.
Edición de solicitud con servicio asociado	400, fecha original conservada.
Edición de servicio con fecha inválida	400, fecha original conservada.
Fin de mes y bisiesto	`minusMonths(6)` : 2025-11-30, 2024-02-29 y 2023-02-28 según el caso.
Prueba de contexto Spring Boot	`mvnw -q test` correcto.

Los registros temporales fueron eliminados. Las cantidades finales quedaron en 12 solicitudes y 13 servicios, sin modificar los registros históricos existentes.

11.14 Conclusión

Las fechas de Solicitud y Servicio ahora se validan en el frontend y en el backend, utilizan meses calendario reales, respetan la relación Solicitud–Servicio y muestran los errores encima de cada campo con el sistema visual existente. No se crearon DTO, no se cambiaron tablas ni se alteraron los registros históricos.